

SupportPilot – AI Ticket Resolution Agent

Milestone 3 – Complete Process & Integration Guide

Weeks 5–6: Multi-Agent Workflows & Integrations

This document explains Milestone 3 from the beginning, including exactly how it connects to Milestone 2, the multi-agent workflow, Jira integration, email automation, database changes, APIs, frontend screens, testing flow, and the final end-to-end process. The structure follows the supplied SupportPilot PDF.

Source basis: The supplied project document defines Milestone 3 as multi-agent architecture, Jira ticket management, and email automation. Its workflow and architecture diagrams show the connection from RAG into the multi-agent investigation workflow, then automated resolution, escalation, Jira, email, and analytics.

1. Where Milestone 3 Starts

Milestone 3 does not replace Milestones 1 and 2. It continues their output.

Milestone 1: Ticket intake → classification → category/sub-category → severity → priority.

Milestone 2: Current ticket → knowledge-base search → semantic/vector retrieval → RAG context → generated troubleshooting resolution.

Milestone 3: Takes the classified ticket and the knowledge/RAG context from Milestone 2 and places them inside a coordinated multi-agent workflow, then connects the result to Jira and email automation.

2. Exact M2 → M3 Connection

The most important connection is that Milestone 3 should consume the outputs already produced by Milestone 2 rather than creating a second independent RAG system.

Milestone 2 produces the retrieved knowledge articles, source/context information, and troubleshooting-generation context. Milestone 3 passes that information into the Knowledge Retrieval Agent and Resolution Agent through the Multi-Agent Orchestrator.

The supplied PDF's Milestone 3 mockup shows the current ticket flowing into a multi-agent workflow containing Diagnosis, Retrieval, Resolution, and Escalation stages, while the integration side shows Jira and Email. The architecture diagram places RAG in the AI intelligence/knowledge path before the agent workflow.

M2 → M3 Flow

```

Ticket
↓
M1 Classification + Severity + Priority
↓
M2 Knowledge Retrieval / RAG
↓
M3 Multi-Agent Orchestrator
↓
Diagnosis Agent → Knowledge Retrieval Agent → Resolution Generation Agent
↓
Confidence / Resolution Validation
■ ■
Auto Resolution Escalation
↓ ↓
Email + Jira Human Support + Jira
↓
Activity Logs / Analytics
```

3. Milestone 3 Modules

Module	Purpose	Main Output
1. Multi-Agent Orchestrator	Coordinates specialized agents and workflow state.	Agent sequence + status
2. Diagnosis Agent	Understands the issue and possible causes.	Diagnosis + confidence
3. Knowledge Retrieval Agent	Uses M2 knowledge/RAG capability to obtain relevant evidence.	Relevant articles/context
4. Resolution Generation Agent	Creates a grounded troubleshooting solution.	Resolution + confidence
5. Escalation Agent	Handles unresolved/low-confidence cases.	Escalation decision
6. Jira Integration	Creates and updates enterprise tickets.	Jira issue/key/status
7. Email Automation	Communicates ticket/resolution/escalation updates.	Email + delivery status
8. Activity Logging	Records every important workflow event.	Audit trail

4. Complete Multi-Agent Process

Step 1 – Receive Existing Ticket

Use the ticket created by Milestone 1. Do not create a duplicate ticket record.

Step 2 – Load M1/M2 Context

Load category, sub-category, severity, priority, ticket description, retrieved knowledge, source references, and prior response context.

Step 3 – Start Workflow

Create a workflow execution and send the ticket to the Multi-Agent Orchestrator.

Step 4 – Diagnosis Agent

Analyze the problem, affected system, likely causes, missing information, and diagnosis confidence.

Step 5 – Knowledge Retrieval Agent

Use the existing M2 knowledge retrieval/RAG layer to obtain relevant troubleshooting guides, SOPs, and enterprise knowledge.

Step 6 – Resolution Agent

Combine ticket + diagnosis + retrieved evidence and generate a step-by-step troubleshooting response.

Step 7 – Validate

Check whether the generated resolution is sufficiently grounded and confident.

Step 8 – Decision

If the issue can be safely resolved, continue to automated response. If unresolved/low confidence, invoke escalation.

Step 9 – Jira

Create/update the Jira issue and synchronize ticket status.

Step 10 – Email

Send the appropriate support communication to the requester.

Step 11 – Log

Record agent execution, Jira action, email action, status changes, and final outcome.

5. Agent Responsibilities

Agent	Input	Work	Output
Orchestrator	Ticket + M1/M2 context	Controls sequence/state	Next agent + workflow status
Diagnosis	Ticket + classification	Identify issue and possible causes	Diagnosis
Knowledge Retrieval	Diagnosis + ticket	Search M2 KB/RAG	Evidence/articles
Resolution	Diagnosis + evidence	Generate grounded steps	Resolution
Escalation	Ticket + failed/low-confidence resolution	Route to human support	Escalation decision

6. Jira Integration – Full Process

The supplied PDF requires Jira integration for ticket management. The database diagram also includes Jira_Tickets connected to Tickets.

Create Jira issue → save Jira ID/key → update status → synchronize status changes → preserve last_updated timestamp.

Event	SupportPilot Action	Jira Action
Ticket ready for enterprise tracking	Create integration request	Create issue
AI working	Set ticket in progress	Set Jira status In Progress
Escalated	Assign support team	Update assignee/team
Resolved	Store resolution	Transition Jira issue to Resolved
Jira changed	Receive/poll status	Sync back to SupportPilot

Recommended Jira API Layer

POST /api/jira/tickets – create Jira issue

GET /api/jira/tickets/:ticketId – get Jira mapping/status

PUT /api/jira/tickets/:ticketId – update Jira issue

POST /api/jira/sync – synchronize Jira status into SupportPilot

7. Email Automation – Full Process

Trigger	Email Type	Recipient	Purpose
Ticket created	Ticket Received	Requester	Confirm ticket creation
AI resolution ready	Resolution	Requester	Send troubleshooting steps
Escalation	Escalation Notice	Requester / support team	Explain human handoff
Ticket resolved	Resolved	Requester	Confirm completion

POST /api/email/ticket-created

POST /api/email/resolution

POST /api/email/escalation

POST /api/email/resolved

GET /api/email/logs/:ticketId

8. Database – M3 Additions

The supplied PDF already defines Users, Tickets, Ticket_Responses, Knowledge_Base, Activity_Logs, Escalations, and Jira_Tickets. For the Milestone 3 implementation, add workflow/agent/email tracking so the multi-agent execution and integrations are auditable.

Table	Important Fields
Agent_Workflows	workflow_id, ticket_id, workflow_status, current_agent, started_at, completed_at, final_confidence
Agent_Executions	execution_id, workflow_id, agent_name, input_data, output_data, status, confidence, started_at, completed_at
Jira_Tickets	jira_id, ticket_id, jira_issue_key, jira_status, last_updated

Email_Logs	email_id, ticket_id, recipient, subject, email_type, status, sent_at
------------	--

9. Activity Log Example

10:32 - Diagnosis Agent identified VPN connection issue
10:32 - Retrieval Agent found 3 relevant knowledge articles
10:33 - Resolution Agent generated troubleshooting steps
10:33 - System recorded resolution confidence
10:34 - Jira issue created/updated
10:34 - Email notification sent
10:35 - Ticket status updated

10. Backend API Structure

POST /api/agent/workflow/start

GET /api/agent/workflow/:ticketId

GET /api/agent/workflow/:ticketId/agents

POST /api/agent/diagnosis

POST /api/agent/retrieve

POST /api/agent/resolve

POST /api/agent/escalate

POST /api/jira/tickets

GET /api/jira/tickets/:ticketId

PUT /api/jira/tickets/:ticketId

POST /api/jira/sync

POST /api/email/ticket-created

POST /api/email/resolution

POST /api/email/escalation

POST /api/email/resolved

GET /api/email/logs/:ticketId

11. Frontend – Milestone 3 Screens

AI Agent Screen – Workflow status, current agent, agent execution timeline, confidence, outputs.

Integrations Screen – Jira Active/Inactive, Email Active/Inactive, connection status, last sync.

Jira Panel – Jira issue key, status, assignee, priority, last synchronization.

Email Panel – Email type, recipient, status, sent time, failure reason if any.

Ticket Detail – M1 classification + M2 retrieved knowledge + M3 agent workflow + Jira + email history.

12. End-to-End Example

User: “VPN is not connecting.”

M1 → Category: Network | Sub-category: VPN Access | Severity: High | Priority: P1

M2 → Retrieves VPN Troubleshooting Guide + firewall/configuration evidence and prepares RAG context.

M3 Orchestrator → starts workflow.

Diagnosis Agent → identifies likely VPN configuration/firewall/authentication causes.

Knowledge Retrieval Agent → uses M2 retrieval/RAG context and returns relevant evidence.

Resolution Agent → generates step-by-step troubleshooting instructions.

Validation → checks resolution quality/confidence.

If acceptable → send automated response + create/update Jira + log activity.

If not acceptable → Escalation Agent → human support team + Jira update + escalation email.

Final → ticket status and all actions are stored for analytics.

13. Week 5 – Development Plan

- Day 1: Define agent interfaces, workflow state, orchestrator and execution model.

- Day 2: Implement Diagnosis Agent.
- Day 3: Connect Knowledge Retrieval Agent to the existing M2 RAG layer.
- Day 4: Implement Resolution Generation Agent.
- Day 5: Implement orchestration, confidence validation, and agent activity logging.

14. Week 6 – Development Plan

- Day 1: Configure Jira authentication, project and issue mapping.
- Day 2: Implement Jira create/update/status synchronization.
- Day 3: Implement email provider, templates and email logging.
- Day 4: Implement ticket-created, resolution, escalation and resolved notifications.
- Day 5: Run complete end-to-end tests and fix integration/workflow failures.

15. Testing Checklist

- Normal ticket enters the M3 workflow successfully.
- Diagnosis Agent receives M1 ticket context.
- Knowledge Retrieval Agent successfully uses M2 RAG output.
- Resolution Agent generates a grounded response.
- Low-confidence/unresolved case reaches Escalation Agent.
- Jira issue is created and mapped to the SupportPilot ticket.
- Jira status updates synchronize correctly.
- Ticket-created email is sent successfully.
- Resolution email is sent successfully.
- Escalation email is sent successfully.
- Every agent/integration event appears in Activity Logs.
- Final ticket status is consistent across SupportPilot and Jira.

16. Milestone 3 Completion Criteria

The supplied project document evaluates Milestone 3 at Week 6 on three core outcomes: multi-agent workflows execute successfully for complex issues; Jira integration functions correctly for ticket management; and email automation delivers support communications successfully.

17. Final Architecture View

```

Users / Support Portal / Email
↓
FastAPI Backend Services
↓
M1 Ticket Classification + Severity
↓
M2 Knowledge Retrieval / RAG
↓
M3 Multi-Agent Orchestrator
    ■■■ Diagnosis Agent
    ■■■ Knowledge Retrieval Agent
  
```

■■■ Resolution Generation Agent
■■■ Escalation Agent
↓
Automated Resolution / Human Escalation
■■■ Jira Integration
■■■ Email Automation
■■■ Ticket Routing
■■■ Activity Logs
↓
Support Dashboard / Analytics

18. Important Implementation Rule

Do not duplicate Milestone 2. M3 should consume and orchestrate the M2 RAG capability. The clean architecture is: M1 provides ticket intelligence → M2 provides grounded knowledge → M3 coordinates agents and enterprise actions. This keeps responsibilities separated and makes the complete system easier to test and maintain.

Source: SupportAI Ticket Agent 2.pdf – supplied project specification, especially pages 4–7 for Milestones 3–4, workflow, architecture and database schema.